

UNIT V

STRINGS, MULTI-THREADING AND SPRING BOOT

STRINGS

A **String** is an object that represents a sequence of characters. defined in the `java.lang` package. Strings are **immutable**, meaning their value cannot be changed once created.

How to Create Strings

1. **Using String literal** (preferred way):

```
String s1 = "Hello World";
```

Stored in the *String Pool* (part of the Java method area memory). If another string with the same content already exists, Java reuses the existing object. It is Memory efficient and Preferred way to create strings.

2. **Using new keyword:**

```
String s2 = new String("Hello World");
```

Always creates a new String object in the **heap**, even if the same value already exists in the String Pool. Not memory-efficient, but allows explicit creation of new objects.

Comparison Example:

```
String s1 = "Hello World";
String s2 = new String("Hello World");
System.out.println(s1 == s2);      // false (different memory references)
System.out.println(s1.equals(s2));  // true (same content)
```

String handling methods:

Method	Description
<code>length()</code>	Returns the length of the string
<code>charAt(int index)</code>	Returns character at given index
<code>substring(int begin, int end)</code>	Returns substring from begin to end-1
<code>equals(String another)</code>	Compares two strings (case-sensitive)
<code>equalsIgnoreCase(String)</code>	Compares strings ignoring case

Method	Description
<code>compareTo(String)</code>	Lexicographically compares strings
<code>toLowerCase()</code>	Converts to lowercase
<code>toUpperCase()</code>	Converts to uppercase
<code>trim()</code>	Removes leading and trailing spaces
<code>replace(char old, char new)</code>	Replaces characters
<code>contains(CharSequence)</code>	Checks if string contains sequence
<code>startsWith(String)</code>	Checks if string starts with specified text
<code>endsWith(String)</code>	Checks if string ends with specified text
<code>indexOf(char)</code>	Returns index of first occurrence
<code>lastIndexOf(char)</code>	Returns index of last occurrence
<code>split(String regex)</code>	Splits string into array by regex
<code>concat(String)</code>	Joins two strings

Examples: String handling methods along with examples.

- **length()**

```
String s = "Hello";  
System.out.println(s.length()); // Output: 5
```

- **charAt(int index)**

```
String s = "Java";  
System.out.println(s.charAt(2)); // Output: 'v'
```

- **substring(int begin, int end)**

```
String s = "Programming";  
System.out.println(s.substring(0, 4)); // Output: "Prog"
```

- **equals(String another)**

```
String s1 = "Test";  
String s2 = "Test";  
System.out.println(s1.equals(s2)); // Output: true
```

- **equalsIgnoreCase(String)**

```
String s1 = "hello";  
String s2 = "HELLO";  
System.out.println(s1.equalsIgnoreCase(s2)); // Output: true
```

- **compareTo(String)**

```
String s1 = "apple";  
String s2 = "banana";  
System.out.println(s1.compareTo(s2)); // Output: negative value
```

- **toLowerCase()**

```
String s = "JAVA";  
System.out.println(s.toLowerCase()); // Output: "java"
```

- **toUpperCase()**

```
String s = "java";  
System.out.println(s.toUpperCase()); // Output: "JAVA"
```

- **trim()**

```
String s = " Hello World ";  
System.out.println(s.trim()); // Output: "Hello World"
```

- **replace(char old, char new)**

```
String s = "banana";  
System.out.println(s.replace('a', '*')); // Output: "b*n*n*"
```

- **contains(CharSequence)**

```
String s = "learning Java";  
System.out.println(s.contains("Java")); // Output: true
```

- **startsWith(String)**

```
String s = "Hello";  
System.out.println(s.startsWith("He")); // Output: true
```

- **endsWith(String)**

```
String s = "world.txt";  
System.out.println(s.endsWith(".txt")); // Output: true
```

- **indexOf(char)**

```
String s = "development";  
System.out.println(s.indexOf('e')); // Output: 1
```

- **lastIndexOf(char)**

```
String s = "banana";  
System.out.println(s.lastIndexOf('a')); // Output: 5
```

- **split(String regex)**

```
String s = "red,green,blue";
```

```
String[] colors = s.split(",");  
System.out.println(colors[1]); // Output: "green"
```

- **concat(String)**

```
String s1 = "Good ";  
String s2 = "Morning";  
System.out.println(s1.concat(s2)); // Output: "Good Morning"
```

Example program on Strings:

```
public class StringExample {  
    public static void main(String[] args) {  
        String s = " Hello Java ";  
        System.out.println("Original: " + s);  
        System.out.println("Length: " + s.length());  
        System.out.println("Trimmed: " + s.trim());  
        System.out.println("Uppercase: " + s.toUpperCase());  
        System.out.println("Substring: " + s.substring(1, 6));  
        System.out.println("Replace: " + s.replace('a', '*'));  
        System.out.println("Starts with ' H': " + s.startsWith(" H"));  
        System.out.println("Index of 'a': " + s.indexOf('a'));  
    }  
}
```

Output:

```
Original:  Hello Java  
Length: 12  
Trimmed: Hello Java  
Uppercase:  HELLO JAVA  
Substring: Hell  
Replace:  Hello J*v*  
Starts with ' H': true  
Index of 'a': 8
```

Multi-line strings:

To create multiline strings in java, one common way is Using \n (Newline Character).

```
String multiline = "This is line 1\nThis is line 2\nThis is line 3";  
System.out.println(multiline);
```

Another way is Using Text Blocks (from Java 15+)

```
String multiline = """
    This is line 1
    This is line 2
    This is line 3
    """;
System.out.println(multiline);
```

Note: Triple double quotes """ denote a text block, and indentation is preserved.

StringBuffer:

StringBuffer is a mutable sequence of characters. Unlike String (which is immutable), StringBuffer can be modified after creation. It is thread-safe (synchronized) — suitable for multithreaded environments.

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" Java");
System.out.println(sb); // Output: Hello Java
```

Commonly Used StringBuffer Methods (with Examples)

- **append(String str):** Appends the string to the buffer.

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");
System.out.println(sb); // Output: Hello World
```

- **insert(int offset, String str):** Inserts a string at the specified position.

```
StringBuffer sb = new StringBuffer("Hello");
sb.insert(2, "Java");
System.out.println(sb); // Output: HeJavallo
```

- **replace(int start, int end, String str):** Replaces characters between start and end with the given string.

```
StringBuffer sb = new StringBuffer("Hello World");
sb.replace(6, 11, "Java");
```

```
System.out.println(sb); // Output: Hello Java
```

- **delete(int start, int end):** Deletes characters from start to end-1.

```
StringBuffer sb = new StringBuffer("Hello Java");  
sb.delete(5, 10);  
System.out.println(sb); // Output: Hello
```

- **reverse():** Reverses the characters in the buffer.

```
StringBuffer sb = new StringBuffer("abcd");  
sb.reverse();  
System.out.println(sb); // Output: dcba
```

- **capacity():** Returns the current capacity (default is 16 + string length).

```
StringBuffer sb = new StringBuffer("Hi");  
System.out.println(sb.capacity()); // Output: 18
```

- **ensureCapacity(int minimumCapacity):** Increases capacity if needed.

```
StringBuffer sb = new StringBuffer();  
sb.ensureCapacity(50);  
System.out.println(sb.capacity()); // Output: 50 or more
```

- **length():** Returns the number of characters in the buffer.

```
StringBuffer sb = new StringBuffer("Java");  
System.out.println(sb.length()); // Output: 4
```

StringBuilder:

StringBuilder is a mutable, growable sequence of characters (just like StringBuffer). It is faster than StringBuffer, but not thread-safe and ideal for single-threaded applications where performance matters.

```
StringBuilder sb = new StringBuilder("Hello");  
sb.append(" Java");  
System.out.println(sb); // Output: Hello Java
```

Commonly used StringBuilder Methods (with Examples)

- **append(String str):** Adds a string at the end.

```
StringBuilder sb = new StringBuilder("Hello");  
sb.append(" World");  
System.out.println(sb); // Hello World
```

- **insert(int offset, String str):** Inserts a string at the given index.

```
StringBuilder sb = new StringBuilder("Java");  
sb.insert(2, "Cool");  
System.out.println(sb); // JaCoolva
```

- **replace(int start, int end, String str):** Replaces characters between start and end - 1.

```
StringBuilder sb = new StringBuilder("Hello Java");  
sb.replace(6, 10, "World");  
System.out.println(sb); // Hello World
```

- **delete(int start, int end):** Deletes characters from start to end - 1.

```
StringBuilder sb = new StringBuilder("DeleteThis");  
sb.delete(6, 10);  
System.out.println(sb); // Delete
```

- **reverse():** Reverses the content.

```
StringBuilder sb = new StringBuilder("ABCD");  
sb.reverse();  
System.out.println(sb); // DCBA
```

- **capacity():** Returns current capacity (default is 16 + initial string length).

```
StringBuilder sb = new StringBuilder("Hi");  
System.out.println(sb.capacity()); // Output: 18
```

- **ensureCapacity(int minimumCapacity):** Ensures the capacity is at least as specified.

```
StringBuilder sb = new StringBuilder();  
sb.ensureCapacity(50);  
System.out.println(sb.capacity()); // Output: >= 50
```

- **length() :** Returns the number of characters.

```
StringBuilder sb = new StringBuilder("Python");  
System.out.println(sb.length()); // 6
```

MULTI-THREADING:

- Multithreading is a process of executing multiple threads simultaneously. Each thread is a lightweight sub-process.
- Java provides built-in support for multithreading using the Thread class or implementing the Runnable interface.
- Multithreading is useful for Efficient CPU usage, Better performance for I/O-heavy or background tasks and Simultaneous execution of independent tasks.

Creating a Thread in Java:

1. By Extending the Thread Class:

In Java, a thread can be created by extending the *Thread* class, and the *run()* method is overridden to include the code that will be executed by the new thread.

Always use *start()* to begin a new thread, not *run()* directly.

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }
}
public class TestThread {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start(); // Starts the thread
    }
}
```

2. By Implementing the Runnable Interface

In Java, a thread can also be created by implementing the *Runnable* interface. This approach is preferred when another class needs to be extended, as multiple inheritance with classes is not supported in Java.

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread is running...");
    }
}
public class TestRunnable {
    public static void main(String[] args) {
        Thread t1 = new Thread(new MyRunnable());
        t1.start(); // Starts the thread
    }
}
```


Main Thread:

The main thread is the first thread that starts when a Java program begins. It is created automatically by the Java Virtual Machine (JVM). The main () method runs inside the main thread.

```
class MainThreadExample {  
    public static void main(String[] args) {  
        // Gets reference to current (main) thread  
        Thread t = Thread.currentThread();  
        System.out.println("Thread Name: " + t.getName());  
        System.out.println("Thread Priority: " + t.getPriority());  
    }  
}
```

Output:

```
Thread Name: main  
Thread Priority: 5
```

Creating Multiple Threads:

In Java, multiple threads can be created and run, with each thread being executed independently and concurrently.

Example:

```
class MyRunnable implements Runnable {  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("From Runnable: " + i);  
        }  
    }  
}  
  
public class MultiRunnableDemo {  
    public static void main(String[] args) {  
        Thread t1 = new Thread(new MyRunnable());  
        Thread t2 = new Thread(new MyRunnable());  
  
        t1.start(); // Starts first thread
```

```
t2.start(); // Starts second thread
}
}
```

Expected Output:

```
From MyThread: 1
From MyThread: 1
From MyThread: 2
From MyThread: 2
...
```

Note: Output may vary because threads run concurrently and their execution order depends on the JVM thread scheduler.

Thread Priorities:

- Thread priority in Java helps the thread scheduler decide the order of thread execution. Each thread is assigned a priority, which is an integer value between 1 and 10.
- Priority Constants

Constant	Value	Description
Thread.MIN_PRIORITY	1	Lowest priority
Thread.NORM_PRIORITY	5	Default (normal) priority
Thread.MAX_PRIORITY	10	Highest priority

- Setting Thread Priority: use the `setPriority(int priority)` method to set a thread's priority.

```
Thread t = new Thread();
t.setPriority(7); // Sets priority to 7
```

- Getting Thread Priority: retrieve a thread's priority using `getPriority()` method.

```
int priority = t.getPriority();
System.out.println("Priority: " + priority);
```

- Example:

```
public class PriorityExample {
    public static void main(String[] args) {
        Thread t1 = new Thread() ->
            System.out.println("Thread 1 is running");
        Thread t2 = new Thread() ->
```

```
        System.out.println("Thread 2 is running"));

        t1.setPriority(Thread.MIN_PRIORITY); // 1
        t2.setPriority(Thread.MAX_PRIORITY); // 10

        System.out.println("Priority of t1: " + t1.getPriority());
        System.out.println("Priority of t2: " + t2.getPriority());

        t1.start();
        t2.start();
    }
}
```

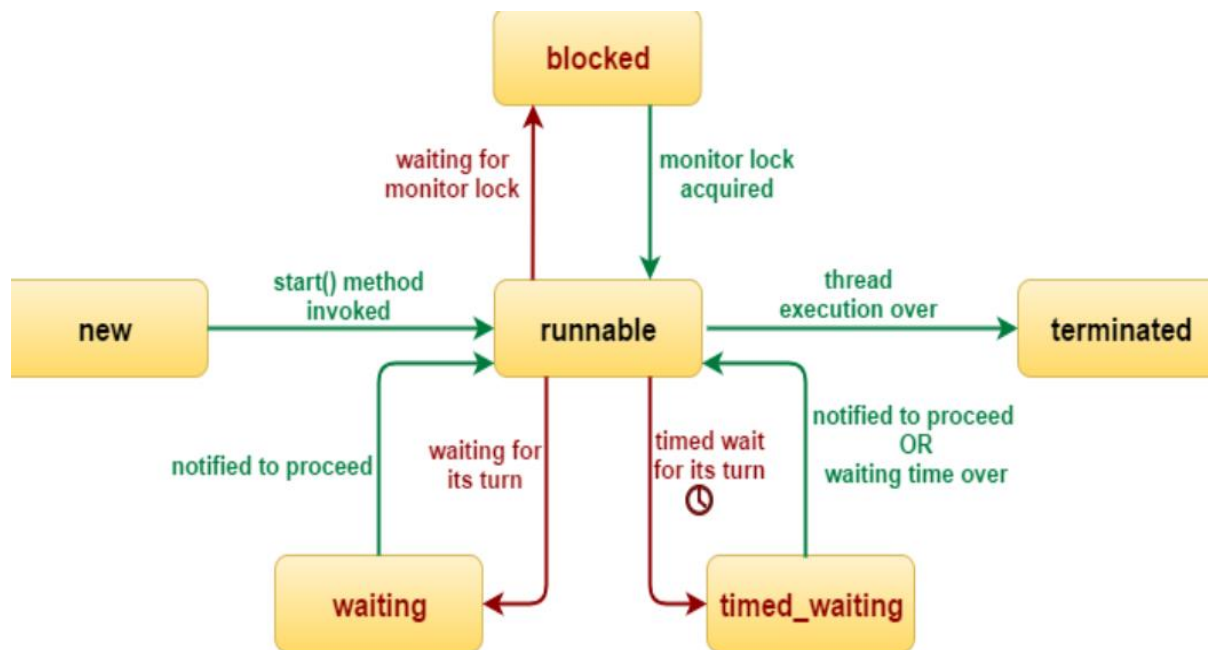
- **Output:**

```
Priority of t1: 1
Priority of t2: 10
Thread 2 is running
Thread 1 is running
```

Thread Life Cycle

The **Thread Life Cycle** in Java refers to the various states a thread goes through during its execution. Here are the main states:

- **New:** A newly created thread object instance on which the start() method has not yet been invoked is in the new state.
- **Runnable:** A thread in new state enters the runnable state when the Thread.start() method is invoked on it.
- **Blocked:** A running thread may enter the blocked state as it waits for a monitor lock to be freed. It may also be blocked as it waits to reenter a monitor lock after being asked to wait using the Thread.wait() method.
- **Waiting:** A thread enters the waiting state when it is made to wait for a go-ahead signal to proceed.
- **Timed_Waiting:** A thread which is waiting as it has been specifically 'instructed' to wait for a specified waiting time (Thread.sleep(2000)) is in a timed_waiting state.
- **Terminated:** A thread enters its 'final resting' state or terminated state when it has finished executing the logic specified in its run() method.



To provide a standard naming and reference for individual thread states, Java language designers have defined an *enum* named *java.lang.Thread.State* which has the following constants defined (each one named after the thread state it refers to) -

- `ThreadState.BLOCKED`
- `ThreadState.NEW`
- `ThreadState.RUNNABLE`
- `ThreadState.TERMINATED`
- `ThreadState.TIMED_WAITING`
- `ThreadState.WAITING`

Some useful methods in Multi-Threading:

- **sleep():** Pauses the currently executing thread for a specified time (in milliseconds).

Syntax: `Thread.sleep(milliseconds);`

Example:

```
Thread.sleep(1000);
```

```
// Pauses for 1 second and it throws InterruptedException
```

- **isAlive():** Checks if a thread is still running or alive (i.e., not terminated). Returns true if thread is running or ready to run, false if terminated.

Syntax: `Thread.isAlive();`

Example:

```
if (t1.isAlive()) {  
    System.out.println("Thread is still running.");  
}
```

- **join():** Waits for a thread to finish execution. Used when one thread needs to wait for another to complete.

Syntax: `Thread.join();` // Throws InterruptedException

Example: `t1.join();` // Main thread waits for t1 to finish

- Simple Example for above methods

```
class MyThread extends Thread {  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(i);  
            try {  
                Thread.sleep(500);  
            } catch (InterruptedException e) {  
                System.out.println(e);  
            }  
        }  
    }  
}  
  
public class Main {  
    public static void main(String[] args) throws InterruptedException  
    {  
        MyThread t1 = new MyThread();  
        t1.start();  
  
        System.out.println("Is Alive: " + t1.isAlive());  
        t1.join(); // Waits for t1 to complete  
        System.out.println("After Join, Is Alive: " + t1.isAlive());  
    }  
}
```

Thread Synchronization:

- If multiple threads want to execute simultaneously in some cases we may get data inconsistency problem. To overcome data inconsistency problem we use Thread synchronization.
- In java Synchronization can be achieved by using *synchronized* keyword.
- *synchronized* keyword is applicable for only methods and blocks but not for classes and variables.
- General form of synchronized method and block.

```
synchronized return_type method_name(arguments)
{
}

synchronized(object)
{
}
```

- In java every object has two areas synchronized area and non-synchronized area.
- In multithreading at a time only one thread can execute synchronized area, in this scenario thread acquires the lock of object. Once thread completes execution then it releases lock. The released lock is allotted to another thread by Thread Scheduler.
- While executing one thread executing synchronized area the remaining threads are in waiting state until current thread completes its execution. The non-synchronized area can execute multiple threads simultaneously.

Example:

```
class Table
{
    synchronized void printTable(int n)
    {
        System.out.println("Multiplication table for :"+n);
        for(int i=1;i<=5;i++){
            System.out.println(n+"*"+i+"="+n*i);
            try{
                Thread.sleep(500);
            }
        }
    }
}
```

```
        catch(Exception e){
            System.out.println(e);
        }
    }
    System.out.println("Thread execution completed\n");
}

class MyThread11 extends Thread
{
    Table t;
    int n;
    MyThread11(Table t,int n){
        this.t=t;
        this.n=n;
    }
    public void run(){
        t.printTable(n);
    }
}

class ThreadSynchronization1{
    public static void main(String args[]){
        Table obj = new Table();//only one object
        MyThread11 t1=new MyThread11(obj,10);
        MyThread11 t2=new MyThread11(obj,20);
        t1.start();
        t2.start();
    }
}
```

Output:

```
Multiplication table for :10 10*1=10
10*2=20
10*3=30
10*4=40
```

```
10*5=50
Thread execution completed

Multiplication table for :20 20*1=20
20*2=40
20*3=60
20*4=80
20*5=100
Thread execution completed
```

Thread Communication (or) Inter Thread Communication:

- Inter-thread communication is the process where multiple threads cooperate and share resources efficiently using wait-notify mechanisms. It helps in avoiding polling and allows threads to communicate with each other via method calls on shared objects.
- Key methods of Thread Communication are

Method	Description
wait()	Causes current thread to wait until another thread calls <code>notify()</code> or <code>notifyAll()</code> .
notify()	Wakes up a single thread waiting on the object's monitor.
notifyAll()	Wakes up all threads waiting on the object's monitor.

- These methods must be called within a synchronized block or method.
- Example:

```
class NotifyThread extends Thread
{
    int total;
    public void run()
    {
        System.out.println("NotifyThread starts updation");

        synchronized(this)
        {
            for(int i=1;i<=100;i++)
                total=total+i;
            System.out.println("NotifyThread giving notification");
            this.notify();
        }
    }
}
```



```
    }  
    }  
}  
class WaitNotifyDemo  
{  
    public static void main(String s[])throws InterruptedException  
    {  
        NotifyThread t1=new NotifyThread();  
        t1.start();  
        synchronized(t1)  
        {  
            System.out.println("Main thread going to waiting state");  
            t1.wait();  
        }  
        System.out.println("Main thread recieves notification");  
        System.out.println("Total is:"+t1.total);  
    }  
}
```

OutPut:

Main thread going to waiting state

NotifyThread starts updation

NotifyThread giving notification

Main thread recieves notification

Total is:5050

Suspending and Resuming Threads:

- To suspend, resume and stop threads we use suspend(), resume() and stop() methods .These methods are called as deprecated methods and not recommended to use in multithreading.

- The general form of these methods

```
final void suspend( )
```

```
final void resume( )
```

```
void stop()
```

- Example:

```
class Thread1 extends Thread  
{  
    public void run(){  
        try  
        {
```

```
        for(int i=1;i<=3;i++)
        {
            sleep(1000);
            System.out.println("this is Thread1 :"+i);
        }
    }
    catch(Exception e){}
}
}
class SuspendResumeDemo
{
    public static void main(String args[])throws Exception
    {
        Thread1 t1=new Thread1();
        t1.start();
        t1.suspend();//its suspends the current thread.
        for(int j=0;j<=3;j++)
        {
            Thread.sleep(1000);
            System.out.println("I am main thread: "+j);
        }
        t1.resume();//its resume the suspended thread.
    }
}
```

Output:

```
I am main thread: 1
I am main thread: 2
I am main thread: 3
this is Thread1 :1
this is Thread1 :2
this is Thread1 :3
```

Introduction to Spring Boot

Spring Boot is a Java-based framework used to create stand-alone, production-ready Spring applications with minimal configuration.

Key Features:

- Auto-configuration
- Embedded servers (Tomcat, Jetty)
- Starter dependencies
- Production-ready with Actuator
- No XML configuration required

Advantages:

- Reduces development time
 - Simplifies deployment
 - Easy integration with Spring ecosystem
 - Provides default configurations
-

Basic Setup and Configuration of Spring Boot Project

Steps to Setup:

1. Create Project:

- Use [Spring Initializr](https://start.spring.io/) (<https://start.spring.io/>)
- Choose: Project (Maven/Gradle), Language (Java), Spring Boot version
- Add dependencies like Spring Web, Spring Boot DevTools

2. Directory Structure:

```
src/
├── main/
│   ├── java/
│   │   └── com.example.demo/
│   │       └── DemoApplication.java
│   └── resources/
│       └── application.properties
```

3. Configuration:

- application.properties (in resources folder)

Example:

```
server.port=8081
```

4. Run Project:

- Using IDE: Run DemoApplication.java
- Using Terminal:

```
mvn spring-boot:run
```

Creating and Running a Simple RESTful Web Service

Step-by-step:

1. Create Main Class:

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

2. Create REST Controller:

```
@RestController
public class HelloController {
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello, Spring Boot!";
    }
}
```

3. Run and Test:

- Run the app
- Open browser/Postman:

```
http://localhost:8080/hello
```

Output:

```
Hello, Spring Boot!
```